# DICS: Data-Informed Centroid Splitting for Decision Tree Classifiers

MD Saifur Rahman Mazumder* Feng Yu†
Department of Mathematical Sciences
University of Texas at El Paso

**Abstract**

Decision tree–based models are widely used in machine learning due to their interpretability and strong empirical performance. However, training decision trees can be computationally expensive, particularly for large and high-dimensional datasets, largely due to the exhaustive search over candidate splits at each node. To improve computational efficiency, we propose *Data-Informed Centroid Splitting (DICS)*, a clustering-based framework that constructs a compact and informative set of candidate splits using data-driven priors. By incorporating class-aware structure, DICS significantly reduces the split search space for *classification tasks* while preserving predictive performance. We further provide theoretical analysis showing that under the stated assumptions, DICS does not degrade the performance of classification trees compared to exhaustive split search. DICS can be incorporated into classification trees, random forests, and gradient-boosting models. Extensive experiments demonstrate that DICS achieves comparable accuracy while substantially reducing training time across synthetic and benchmark datasets, highlighting the benefit of integrating data-informed priors into split selection for scalable classification tree learning.

## 1 Introduction

Data-driven methods have become fundamental across a wide spectrum of real-world applications, where systems must extract insights from large and complex datasets to make reliable decisions. Tasks such as spam filtering, targeted advertising, fraud detection, and scientific discovery all depend on models that can capture intricate patterns while remaining computationally efficient. Among the broad family of machine learning techniques, decision tree–based models are particularly appealing due to their interpretability, flexibility, and strong empirical performance. Ensemble variants, including random forests [1] and gradient boosting machines [2], are especially effective, as they can model nonlinear relationships and higher-order feature interactions while maintaining relatively low computational overhead. Their hierarchical structure further enables them to handle heterogeneous data and provide interpretable decision rules, making them a key component of modern scalable learning systems.

*Decision tree* (DT) [3], also known as *Classification and Regression Tree (CART)*, is a simple and interpretable model that produces predictions by recursively partitioning the feature space through a sequence of binary decisions. Despite its conceptual simplicity, training a decision tree can be computationally demanding. At each node, the algorithm evaluates a large number of candidate split points, which are typically derived from sorted feature values, and selects the optimal split

*msmazumder@miners.utep.edu
†Corresponding author. fyu@utep.edu

according to a predefined criterion. This *exhaustive search process* is repeated independently across nodes, leading to significant computational costs as the number of samples, features, or tree depth increases.

To address these challenges, a substantial body of work has focused on improving the efficiency and scalability of tree construction. Early approaches such as SLIQ [4] and RainForest [5] employ presorting techniques, breadth-first growth strategies, and compact data representations to decouple scalability from split evaluation. QUEST [6] addresses a related problem by enabling fast tree construction while reducing variable-selection bias. [7] proposed dynamic split-point selection strategies that restrict the number of candidate thresholds for continuous features; however, these methods often rely on restrictive assumptions and may not scale well in high-dimensional settings. In the context of ensemble learning, Extremely Randomized Trees [8] further reduce computational cost by introducing randomness in both feature selection and split thresholds, thereby avoiding exhaustive search. More recent systems, such as XGBoost [9] and LightGBM [10], achieve substantial efficiency gains through approximate split finding, sparsity-aware computation, and histogram-based binning techniques. Collectively, these developments highlight that split selection remains a primary computational bottleneck in tree-based learning.

In this paper, we address the computational challenges of *Classification Trees* by exploiting structural information in the data to guide split selection. In classification tasks, the underlying data patterns provide valuable structural information that can guide split selection. Motivated by this observation, we propose ***Data-Informed Centroid Splitting (DICS)***, a method that leverages data-driven priors through clustering to construct a significantly reduced set of candidate splits, thereby substantially accelerating the tree training process.

Beyond conventional greedy tree induction, another line of work considers globally optimized sparse decision trees, where branch-and-bound and related search strategies optimize the complete tree structure rather than selecting splits greedily [11, 12, 13, 14]. Within this setting, McTavish et al. [15], for example, use a boosted tree ensemble as a reference model to identify informative thresholds for accelerating the optimization of sparse decision trees. While DICS shares the general motivation of restricting the continuous split space, it derives the candidates directly from the clustering structure of the input data without requiring an auxiliary predictive model. Clustering has also been incorporated more directly into tree construction, Clus-DTI [16] uses clustering to decompose a classification problem into simpler subproblems before inducing decision trees, while BDTKS [17] recursively applies K-means within a multivariate binary tree and converts centroid-based separations into hyperplane tests. In contrast, DICS uses clustering only to construct a reusable set of univariate feature–threshold candidates, leaving the underlying greedy tree objective unchanged and allowing the same candidate-generation strategy to be used with classification trees, random forests, and gradient-boosting models.

In summary, our contributions are as follows:

1. We propose *Data-Informed Centroid Splitting (DICS)*, a clustering-driven approach for constructing compact candidate split sets in classification trees.
2. We provide theoretical analysis showing that decision trees built with DICS preserve predictive performance compared to standard exhaustive split search.
3. We conduct extensive experiments demonstrating that DICS achieves comparable accuracy while significantly improving computational efficiency.

## 2 Preliminaries

### 2.1 Classification trees

We consider a multi-class classification problem. Let $\mathbf{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{P \times N}$ denote the feature matrix and $\boldsymbol{y} \in [K]^N$ the corresponding labels, where $N$ is the number of samples, $P$ is the number of features, and $K$ is the number of classes. Formally, a classification tree can be defined by $T(\boldsymbol{x}; \boldsymbol{\theta}) = \sum_{m=1}^{M} c_m \mathbb{I}(\boldsymbol{x} \in R_m)$ where $\boldsymbol{\theta} = \{(c_m, R_m)\}_{m=1}^{M}$ is the parameter, $R_m$ denotes the $m$-th rectangle region, $c_m \in [K]$ is the predicted class assigned to $R_m$, and $M$ is the total number of regions. To fit the model, we minimize the following empirical risk:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell(y_i, T(\boldsymbol{x}_i; \boldsymbol{\theta})) = \sum_{m=1}^{M} \sum_{\boldsymbol{x}_i \in R_m} \ell(y_i, c_m). \tag{1}$$

Solving (1) typically involves two stages: (1) learning the discrete tree structure $\{R_m\}_{m=1}^{M}$, and (2) estimating the optimal labels $c_m$ given the partition $\{R_m\}_{m=1}^{M}$. However, the first step makes (1) non-differentiable and computationally challenging. In fact, finding the optimal data partition induced by a decision tree is NP-complete [18].

Each region $R_m$ corresponds to a leaf node and is formed by a sequence of recursive binary splits. Equivalently, every $R_m$ can be expressed as the intersection of splitting constraints along the path from the root to the corresponding leaf: $R_m = \bigcap_{t \in \mathcal{P}_m} \left\{ \boldsymbol{x} \in \mathbb{R}^P : x_{f_t} \in S_t \right\}$, where $\mathcal{P}_m$ denotes the set of internal nodes along the path to leaf $m$, $f_t$ is the feature selected at node $t$, and $S_t \in \{(-\infty, \tau_t), [\tau_t, \infty)\}$ specifies the split induced by threshold $\tau_t$. A feature-threshold pair $(f, \tau)$ defines a split that partitions the index set $[N] = \{1, \ldots, N\}$ into two subsets:

$$\mathcal{I}_L = \{i \in \mathcal{I} : \mathbf{X}_{f,i} < \tau\}, \quad \mathcal{I}_R = \{i \in \mathcal{I} : \mathbf{X}_{f,i} \geq \tau\}.$$

The search for the optimal splits is typically performed using a greedy, top-down strategy. Starting from the root node, all candidate splits are evaluated at each node. Specifically, for each feature $f$, the samples are first sorted according to their values along that feature, and candidate thresholds are chosen as the midpoints between consecutive distinct values. The quality of a split $(f, \tau)$ is measured by the weighted post-split impurity:

$$Q(f, \tau) = \frac{|\mathcal{I}_L|}{N} G(\mathcal{I}_L) + \frac{|\mathcal{I}_R|}{N} G(\mathcal{I}_R). \tag{2}$$

Here, $G(\mathcal{I})$ denotes the impurity of a node indexed by $\mathcal{I}$. Common choices include the Gini index and entropy (deviance). To compute $G(\mathcal{I})$, we first estimate the empirical class distribution within $\mathcal{I}$ as $\hat{\pi}_c = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbb{I}(y_i = c)$. The Gini index is defined as $G(\mathcal{I}) = \sum_{c=1}^{K} \hat{\pi}_c (1 - \hat{\pi}_c) = 1 - \sum_{c=1}^{K} \hat{\pi}_c^2$ [19, 3], while the entropy is given by $G(\mathcal{I}) = -\sum_{c=1}^{K} \hat{\pi}_c \log \hat{\pi}_c$ [20]. This impurity-based criterion underlies the standard greedy split selection in classification trees and is widely used in tree-based methods [3].

### 2.2 Ensemble trees

Although a single decision tree offers strong interpretability, it typically suffers from high variance, instability, and sensitivity to noise in the training data. In practice, ensemble tree methods are therefore often preferred. These approaches combine multiple decision trees to construct a stronger predictive model. The main types of ensemble trees include *bagging* (or *bootstrap aggregating*), *Random Forest* (RF, a specific instance of bagging) [1], *boosting*, and etc.

For classification tasks, an ensemble tree typically makes predictions via majority voting: $\hat{y}(\boldsymbol{x}) = \arg\max_{c\in[K]} \sum_{b=1}^{B} \mathbb{I}(T_b(\boldsymbol{x}) = c)$, where $\{T_b\}_{b=1}^{B}$ are the individual trees. In bagging, each tree $T_b$ is trained on a bootstrap sample $(\mathbf{X}^{*b}, \boldsymbol{y}^{*b})$. Random Forest further reduces variance by *de-correlating* trees, typically by restricting each tree to a random subset of features (of size $\lfloor\sqrt{P}\rfloor$ for classification). In boosting, trees are constructed sequentially, with each new tree designed to correct the errors made by the previous ones. Among these approaches, gradient boosting methods such as XGBoost [9] and LightGBM [10] are widely recognized for achieving strong predictive performance on tabular data.

### 2.3 Computational bottleneck

As discussed in earlier sections, the most challenging and computationally demanding component of a Decision Tree (DT) is learning the discrete tree structure, which is equivalent to searching for optimal splits (i.e., feature–threshold pairs). The standard approach adopts a greedy, top-down strategy: at each node, it ***exhaustively*** evaluates every split out of $(N-1)P$ candidates, corresponding to $P$ features and up to $(N-1)$ candidate thresholds per feature (typically taken as midpoints between consecutive sorted sample values). As a result, the overall training cost scales rapidly with the sample size, tree depth, and ensemble size.

This ***large candidate space*** is the main computational bottleneck of DT training. To alleviate this issue, a popular alternative is the histogram-based method [9, 21, 10]. This approach discretizes each feature into a finite number of bins and constructs feature histograms, allowing the optimal split to be found by scanning histogram bins instead of raw data values. However, this binning strategy introduces additional approximation errors. Specifically, multiple distinct feature values are merged into the same bin, and candidate splits are restricted to bin boundaries only. This leads to two main drawbacks: (1) *quantization error* due to coarse discretization of continuous features, and (2) a *trade-off* between efficiency and accuracy that depends on the number of bins used. In particular, using fewer bins improves computational efficiency but increases information loss, while using more bins reduces quantization error at the cost of higher computational and memory overhead.

Nevertheless, both exhaustive search and histogram-based methods lack prior guidance on which splits are likely to be informative. This motivates our approach, which (i) constructs a compact, data-informed set of candidate splits, and (ii) enforces a fixed per-node evaluation budget, ensuring that the split-search cost remains predictable and controlled.

## 3 Methodology

In this section, we introduce a novel approach, termed *Data-Informed Centroid Splitting (DICS)*, which precomputes a compact, data-informed set of candidate thresholds for use in classification trees (see Section 3.1). This strategy directly addresses the split-search bottleneck without modifying the underlying tree objective. We then describe how DICS can be integrated into standard classification tree-based frameworks, including classical decision trees, random forests [1], and gradient boosting systems, in Section 3.2. We further analyze the computational complexity of DICS in Section 3.3.

### 3.1 Data-Informed Centroid Splitting

A fundamental assumption in classification is that samples belonging to the same class are generally close to each other in the feature space. This principle underlies many methods, such as k-nearest neighbors and linear discriminant analysis [22, 23, 24]. Consequently, in ideal scenarios, one might expect the classification boundaries to roughly align with clusters formed by the data even without in the absence of label information. Although this alignment may not hold exactly in practice, clustering

results can still provide valuable guidance and serve as useful prior information. A simple illustrative example is shown in the left panel of Figure 1: we generate $N = 600$ two-dimensional samples for two classes with $\boldsymbol{x}|y = 0 \sim \mathcal{N}((-1, 0), \Sigma)$ and $\boldsymbol{x}|y = 1 \sim \mathcal{N}((1, 1), \Sigma)$ and $\Sigma = [0.25, 0.1; 0.1, 0.25]$. In this setting, the optimal Bayes decision boundary (labeled as classification boundary) is well approximated by the separator obtained from applying K-means clustering to the data.

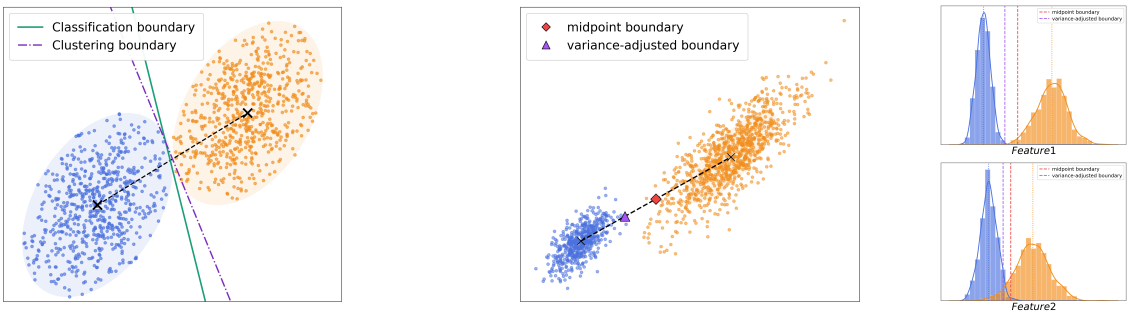



Figure 1: Illustrative example: (left) classification vs. clustering boundaries; (right) midpoint vs. variance-adjusted boundaries.

Moreover, clustering can typically be performed efficiently, for instance, via K-means [25]. Motivated by the observed alignment between clustering boundaries and classification decision boundaries, as well as the computational effectiveness of K-means, we propose a novel clustering-guided approach for generating candidate splits. Specifically, let $\{\boldsymbol{\mu}_c\}_{c=1}^{K} \subset \mathbb{R}^P$ be the centroids obtained from applying K-means clustering to the feature matrix $\mathbf{X}$. Each centroid $\boldsymbol{\mu}_c$ represents a dominant mode of the underlying data distribution, providing a coarse yet stable summary of where the data concentrates.

We propose **Data-Informed Centroid Splitting (DICS)** approach, which leverages clustering boundaries as candidate classification boundaries, further yielding splits for tree-based models. In particular, for any pair of clusters $c_1$ and $c_2$, a boundary can be defined based on their centroids. For a given feature $f$, this induces a candidate split of the form $\zeta(\mu_{c_1}^{(f)}, \mu_{c_2}^{(f)})$. However, in the absence of labels, there is no unique "correct" boundary in clustering. In K-means, a point $\boldsymbol{x}$ is assigned to the cluster with the nearest centroid, i.e., $\arg\min_{1 \le c \le K} \|\boldsymbol{x} - \boldsymbol{\mu}_c\|$. The boundary between two clusters is thus given by the set of points satisfying $\|\boldsymbol{x} - \boldsymbol{\mu}_{c_1}\| = \|\boldsymbol{x} - \boldsymbol{\mu}_{c_2}\|$, which induces a Voronoi partition of the space [26]. Under this construction, the corresponding one-dimensional split is given by the midpoint, $\zeta(\mu_{c_1}^{(f)}, \mu_{c_2}^{(f)}) = \frac{1}{2}(\mu_{c_1}^{(f)} + \mu_{c_2}^{(f)})$. Nevertheless, this midpoint-based boundary may be suboptimal when clusters exhibit different variances. For example, if one class is significantly more dispersed than the other (as illustrated in the right panel of Figure 1), the midpoint can be biased toward the wider cluster. To address this issue, we propose a variance-adjusted boundary in which the distances from the centroids to the boundary are proportional to their standard deviations, i.e. $\frac{\zeta - \mu_{c_1}^{(f)}}{\mu_{c_2}^{(f)} - \zeta} = \frac{\hat{\sigma}_{c_1}}{\hat{\sigma}_{c_2}}$, which implies that $\zeta(\mu_{c_1}^{(f)}, \mu_{c_2}^{(f)}) = \frac{\hat{\sigma}_{c_1}\mu_{c_2}^{(f)} + \hat{\sigma}_{c_2}\mu_{c_1}^{(f)}}{\hat{\sigma}_{c_1} + \hat{\sigma}_{c_2}}$, where $\hat{\sigma}_{c_1}$ and $\hat{\sigma}_{c_2}$ are the standard deviations of feature $f$ within clusters $c_1$ and $c_2$, respectively. This formulation provides a more adaptive and robust candidate split when cluster spreads differ.

We also note that there are $\binom{K}{2} = \frac{K(K-1)}{2}$ possible pairs of centroids, whereas $K$ clusters are present. When two centroids are very close, the boundary between them is unlikely to correspond to a meaningful split in the tree. To eliminate such redundant splits, we retain only $m$ centroid pairs corresponding to the $m$ largest inter-centroid distances $D_{ij} = \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|$. The resulting procedures are summarized in Algorithm 1.

**Algorithm 1** Data-Informed Centroid Splitting (DICS)

**Require:** Feature matrix $\mathbf{X} \in \mathbb{R}^{P \times N}$, maximum pair count $m < \frac{K(K-1)}{2}$.
**Ensure:** Split-candidate dictionary $\mathcal{S}$.

1: Fit K-means on $\mathbf{X}$ with $K$ clusters to obtain centroids $\{\boldsymbol{\mu}_c\}_{c=1}^{K}$
2: Compute pairwise centroid distances: $D_{ij} \leftarrow \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|, \forall i < j$
3: Select $\mathcal{P} \leftarrow$ the $m$ pairs $(i, j)$ with largest $D_{ij}$
4: **for** $f = 1$ **to** $P$ **do**
5: $\quad T_f \leftarrow \left\{ \left(f, \zeta(\mu_i^{(f)}, \mu_j^{(f)})\right) : (i, j) \in \mathcal{P} \right\}$, where $\zeta = (\hat{\sigma}_{c_1}\mu_{c_2}^{(f)} + \hat{\sigma}_{c_2}\mu_{c_1}^{(f)})/(\hat{\sigma}_{c_1} + \hat{\sigma}_{c_2})$.
6: **end for**
7: **return** $\mathcal{S} = \cup_{f=1}^{P}\{T_f\}$.

### 3.2 Cluster-guided (ensemble) trees

The DICS algorithm (Algorithm 1) precalculates candidate splits guided by clustering, and the classification tree is subsequently grown from the resulting split-candidate dictionary $\mathcal{S}$. The corresponding adaptations of the decision tree and random forest, referred to as the *Cluster-Guided Classification Tree* (CGCT) and *Cluster-Guided Random Forest* (CGRF), are summarized in Algorithm 2 (see Section A.1.1 for details) and Algorithm 3 (see Section A.1.2 for details), respectively.

As for gradient boosting machine, we propose **Fast C**lassification **G**radient **B**oosting **M**achine (*FastC-GBM*), which leverages candidate splits generated by DICS and incorporates the Gradient-based One-Side Sampling (GOSS) technique [10] and column (feature) subsampling technique [9, 10]. In particular, when evaluating the quality of a split $(f, \tau)$, the computational cost is further reduced by sampling a subset of samples with small gradient magnitudes and subsampling features. The full procedure of FastC-GBM is summarized in Algorithm 4 (see Section A.1.3 for details).

### 3.3 Implementation and complexity

The candidate splits generated by DICS depend on the results of K-means. Since DICS only requires stable centroids, we adopt mini-batch K-means [27] together with K-means++ initialization [28]. Under this strategy, DICS introduces a one-time computational overhead of order $\mathcal{O}(TKbP)$, where $T$ is the number of iterations, $b$ is the mini-batch size, $K$ is the number of clusters, and $P$ is the feature dimension. This one-time is relative small compared to the split finding stage.

For split finding, DICS requires a computational cost of $\mathcal{O}(mPU)$, where $m$ is the number of centroid pairs and $U$ denotes the number of operations needed to evaluate the quality of a candidate split. In contrast, an exhaustive search incurs a cost of $\mathcal{O}(NPU)$, while histogram-based search reduces this to $\mathcal{O}(HPU)$, where $H$ is the number of bins, typically set to 255 to balance efficiency and accuracy [10]. Consequently, DICS significantly reduces the per-tree training cost to a factor of $m$. When the number of classes $K$ is small (which is often the case), $m = K(K-1)/2$ remains small; for example, $m = 10$ when $K = 5$. When $K$ is large, our sensitivity experiments (see Appendix A.3) show that, in practice, a small value of $m$ is still sufficient to achieve comparable accuracy. We further observe that accuracy remains stable across a range of tree depths ($d$) and mini-batch sizes ($b$).

# 4 Split-Level Stability Analysis

In this section, we provide a theoretical analysis of the proposed DICS algorithm. Our main result in Theorem 1 shows that the gain of the optimal split selected by DICS is close to that of the optimal split selected by a standard decision tree (DT). In particular, the difference between these two gains is bounded by $\mathcal{O}(1/\sqrt{N})$, implying that as the number of samples $N$ increases, the gap vanishes asymptotically.

Let the set of candidate splits of DT be given by $\mathcal{S}_{\mathrm{DT}} = \{(f, \bar{\tau}_i^{(f)}) : i = 1, \ldots, N-1\}$ where $\bar{\tau}_i^{(f)}$ are the midpoints of the sorted observation values along feature $f$. The set of candidate splits generated by DICS is denoted by $\mathcal{S}_{\mathrm{DICS}} = \{(f, \tilde{\tau}_i^{(f)}) : i = 1, \ldots, m\}$ where $\tilde{\tau}_i^{(f)}$ are the thresholds produced by the DICS procedure. For any split $(f, \tau)$, it defines a partition $\mathcal{P}_{f,\tau} = (\mathcal{I}_L, \mathcal{I}_R)$, where $\mathcal{I}_L = \{i \in [N] : \mathbf{X}_{f,i} < \tau\}$ and $\mathcal{I}_R = \{i \in [N] : \mathbf{X}_{f,i} \geq \tau\}$ are the corresponding index sets with sizes $n_L = |\mathcal{I}_L|$ and $n_R = |\mathcal{I}_R|$. The optimal splits for DT and DICS are defined as $(\bar{f}^*, \bar{\tau}^*) = \arg\min_{(f,\tau)\in\mathcal{S}_{\mathrm{DT}}} Q(f, \tau)$ and $(\tilde{f}^*, \tilde{\tau}^*) = \arg\min_{(f,\tau)\in\mathcal{S}_{\mathrm{DICS}}} Q(f, \tau)$, respectively. Consequently, the corresponding gains for DT and DICS are given by $G(\mathcal{I}) - Q(\bar{f}^*, \bar{\tau}^*)$ and $G(\mathcal{I}) - Q(\tilde{f}^*, \tilde{\tau}^*)$, respectively. For notational convenience, we denote $\bar{Q}^* = Q(\bar{f}^*, \bar{\tau}^*)$ and $\tilde{Q}^* = Q(\tilde{f}^*, \tilde{\tau}^*)$.

**Theorem 1.** *For every $f$, let $\bar{\tau}_*^{(f)} = \arg\min_{\tau\in\{\bar{\tau}_i^{(f)}, 1\leq i\leq N-1\}} Q(f, \tau)$ be the optimal threshold for DT and $\tilde{\tau}_*^{(f)} = \arg\min_{\tau\in\{\tilde{\tau}_i^{(f)}, 1\leq i\leq m\}} Q(f, \tau)$ be the optimal threshold for DICS, where $Q(f, \tau)$ is the Gini impurity function. Suppose that $\alpha = \min_f\{\frac{n_L^{(f)}}{N}, \frac{n_R^{(f)}}{N}\} > 0$. Further assume that a density regularity condition holds, i.e., for every feature $f$, the marginal density $p(x^{(f)})$ is bounded above by a constant $M$ in a neighborhood of the optimal split $\tau_*^{(f)}$. Then $\delta = d_H\Big(\mathcal{P}_{f,\bar{\tau}_*^{(f)}}, \mathcal{P}_{f,\tilde{\tau}_*^{(f)}}\Big) = \mathcal{O}_P(\sqrt{N})$. Moreover, the following bound holds:*

$$|\bar{Q}^* - \tilde{Q}^*| < \left(8 + \frac{1}{\alpha(1-\alpha)} \cdot \frac{\delta}{N}\right) \frac{\delta}{N}.$$

*Proof.* The proof is provided in Appendix A.2. □

Here, $d_H(\mathcal{P}_1, \mathcal{P}_2)$ in the above theorem denotes the Hamming distance between two partitions, defined as $d_H(\mathcal{P}_1, \mathcal{P}_2) = |\{i \in [N] : z_i \neq z_i'\}|$ where each partition $\mathcal{P} = (\mathcal{I}_L, \mathcal{I}_R)$ is associated with a label vector $z \in \{L, R\}^N$ defined by $z_i = L$ if $i \in \mathcal{I}_L$ and $z_i = R$ if $i \in \mathcal{I}_R$.

**Remark 1.** The assumption in Theorem 1 states that the data distribution is well-behaved in a neighborhood of the decision boundary. Such a condition is standard in statistical learning theory, as it rules out degenerate cases in which the marginal density vanishes or concentrates excessively near the optimal split. Under this assumption, Theorem 1 shows that the partitions induced by the optimal thresholds of DT and DICS differ on at most $\delta = \mathcal{O}_P(\sqrt{N})$ samples. Although $\delta$ grows with $N$, the difference between the corresponding optimal gains satisfies $\mathcal{O}_P(1/\sqrt{N})$, and therefore converges to zero as $N$ increases. We provide complementary empirical evidence for this split-level behavior in Appendix A.4. This bound is established at the root, where the DICS dictionary is estimated from the same distribution being split; at deeper nodes, the global dictionary is not guaranteed to satisfy the same alignment, and the depth-wise diagnostic in Appendix A.4 (Table 7) shows the cumulative retained gain ratio $R_d$ declining from 100% at the root to 97.18% by depth 8.

**Remark 2.** Theorem 1 only establishes that the gain achieved by DICS is asymptotically close to that of DT, which does not imply that one method is necessarily better than the other, as the relationship between gain and classification accuracy is nontrivial. This theoretical result is also supported by our numerical experiments.

# 5 Numerical Experiments

We evaluate the proposed split-generation method, DICS, across several decision tree–based models, including *(1) standard decision trees (DT), (2) random forests (RF), and (3) gradient boosting systems*. The comparisons are conducted on both synthetic and real-world datasets. All results are obtained on a machine equipped with an Apple M4 Max chip and 64GB of unified memory.

**Baseline Methods.** As relatively few studies integrate clustering into tree construction, we restrict our comparison in the DT setting to BDTKS [17]. We exclude Clus-DTI [16], as it alters the underlying tree objective. Moreover, since BDTKS applies the K-means algorithm recursively at each node, it becomes computationally expensive for large datasets and is not readily compatible with ensemble methods. Therefore, we do not include it in the RF and boosting settings.

**Parameters Setup.** We use the Gini index as the splitting criterion to compute the quality $Q(f, \tau)$. The maximum tree depth is set to $D = 8$ for DT and RF, and $D = 3$ for boosting-based methods. The number of sampled candidate splits is fixed at $k = 100$. For $K \leq 5$, we set the number of centroid pairs to $m = K(K-1)/2$; otherwise, we use $m = 25$. The batch size for mini-batch K-means is set to 512.

| $(N, P)$ | Class | Time (sec)↓ | | | Speedup↑ | Test Acc↑ | | |
|---|---|---|---|---|---|---|---|---|
| | | DT | BDTKS | CGCT | $t_{\mathrm{DT}}/t_{\mathrm{CGCT}}$ | DT | BDTKS | CGCT |
| 10000,1000 | 3 | 2.47 (0.03) | 30.16 | **0.19 (0.02)** | 13.00 | **0.62 (0.00)** | 0.54 | 0.60 (0.00) |
| | 5 | 2.53 (0.01) | 34.36 | **0.25 (0.02)** | 10.12 | **0.54 (0.00)** | 0.39 | 0.52 (0.00) |
| | 10 | 2.52 (0.00) | 35.68 | **0.29 (0.02)** | 8.69 | **0.43 (0.00)** | 0.32 | 0.41 (0.00) |
| 5000,7000 | 3 | 8.36 (0.10) | 29.85 | **0.58 (0.05)** | 14.41 | **0.61 (0.01)** | 0.57 | 0.59 (0.01) |
| | 5 | 8.14 (0.03) | 29.44 | **0.91 (0.04)** | 8.94 | **0.47 (0.01)** | 0.38 | 0.45 (0.00) |
| | 10 | 8.27 (0.02) | 30.93 | **1.04 (0.08)** | 7.95 | **0.31 (0.00)** | 0.27 | **0.31 (0.01)** |
| 20000,5000 | 3 | 26.89 (0.26) | 107.82 | **1.20 (0.07)** | 22.40 | **0.53 (0.00)** | 0.47 | 0.52 (0.00) |
| | 5 | 27.17 (0.04) | 110.24 | **1.77 (0.07)** | 15.35 | **0.47 (0.00)** | 0.38 | 0.46 (0.00) |
| | 10 | 27.98 (0.05) | 114.27 | **1.99 (0.09)** | 14.06 | 0.48 (0.00) | 0.39 | **0.49 (0.00)** |

Table 1: Accuracy and training time (in seconds) for DT, BDTKS, and CGCT in synthetic data. The ratio $t_{\mathrm{DT}}/t_{\mathrm{CGCT}}$ quantifies the training time improvement of CGCT relative to DT.

| $(N, P)$ | Class | Time (sec)↓ | | Speedup↑ | Test Acc↑ | |
|---|---|---|---|---|---|---|
| | | RF | CGRF | $t_{\mathrm{RF}}/t_{\mathrm{CGRF}}$ | RF | CGRF |
| 10000,1000 | 3 | 4.05 (0.13) | **0.36 (0.04)** | 11.25 | **0.64 (0.00)** | **0.64 (0.00)** |
| | 5 | 4.32 (0.02) | **0.41 (0.03)** | 10.54 | **0.56 (0.01)** | **0.56 (0.00)** |
| | 10 | 4.25 (0.04) | **0.53 (0.02)** | 8.02 | **0.45 (0.01)** | **0.45 (0.01)** |
| 5000,7000 | 3 | 13.27 (0.45) | **0.93 (0.10)** | 14.27 | **0.64 (0.00)** | **0.64 (0.01)** |
| | 5 | 13.18 (0.05) | **1.14 (0.05)** | 11.56 | **0.53 (0.00)** | 0.52 (0.00) |
| | 10 | 13.17 (0.04) | **1.66 (0.08)** | 7.93 | **0.36 (0.00)** | **0.36 (0.00)** |
| 20000,5000 | 3 | 44.47 (1.45) | **1.49 (0.04)** | 29.84 | **0.56 (0.00)** | 0.54 (0.00) |
| | 5 | 48.26 (1.07) | **1.67 (0.10)** | 28.90 | **0.49 (0.00)** | **0.49 (0.00)** |
| | 10 | 53.25 (0.60) | **2.09 (0.06)** | 25.48 | **0.52 (0.00)** | **0.52 (0.00)** |

Table 2: Accuracy and training time (in seconds) for RF and CGRF in synthetic data. The ratio $t_{\mathrm{RF}}/t_{\mathrm{CGRF}}$ quantifies the training time improvement of CGRF relative to RF.

| $(N, P)$ | Class | Time (sec)↓ | | | Speedup↑ | | Test Acc↑ | | |
|---|---|---|---|---|---|---|---|---|---|
| | | XGBoost | LightGBM | FastC-GBM | $\rho_1$ | $\rho_2$ | XGBoost | LightGBM | FastC-GBM |
| 10000,1000 | 3 | 1.62 (0.03) | 1.16 (0.06) | **0.32 (0.00)** | 5.06 | 3.62 | **0.68 (0.00)** | **0.68 (0.0)** | 0.65 (0.01) |
| | 5 | 2.66 (0.07) | 1.81 (0.07) | **0.35 (0.00)** | 7.60 | 5.17 | 0.59 (0.01) | **0.60 (0.01)** | 0.59 (0.01) |
| | 10 | 5.03 (0.02) | 3.31 (0.0) | **0.51 (0.01)** | 9.86 | 6.49 | **0.47 (0.01)** | 0.46 (0.01) | 0.45 (0.01) |
| 5000,7000 | 3 | 6.62 (0.06) | 3.09 (0.03) | **0.24 (0.00)** | 27.58 | 12.87 | **0.63 (0.00)** | **0.63 (0.02)** | 0.61 (0.02) |
| | 5 | 11.25 (0.09) | 4.96 (0.07) | **0.28 (0.00)** | 40.18 | 17.71 | **0.51 (0.02)** | 0.50 (0.02) | 0.50 (0.02) |
| | 10 | 18.06 (0.45) | 9.80 (0.33) | **0.36 (0.00)** | 50.17 | 27.22 | **0.35 (0.01)** | **0.35 (0.01)** | 0.34 (0.01) |
| 20000,5000 | 3 | 8.52 (0.30) | 5.28 (0.18) | **0.66 (0.02)** | 12.90 | 8.00 | **0.57 (0.01)** | **0.57 (0.01)** | 0.56 (0.01) |
| | 5 | 15.57 (0.61) | 8.41 (0.24) | **0.48 (0.00)** | 32.44 | 17.52 | **0.51 (0.01)** | **0.51 (0.01)** | 0.49 (0.01) |
| | 10 | 31.41 (0.27) | 14.72 (0.12) | **1.01 (0.02)** | 31.10 | 14.57 | **0.53 (0.01)** | **0.53 (0.01)** | **0.53 (0.01)** |

Table 3: Accuracy and training time (in seconds) for XGBoost, LightGBM and FastC-GBM in synthetic data. The ratios $\rho_1 = t_{\text{XGBoost}}/t_{\text{FastC-GBM}}$ and $\rho_2 = t_{\text{LightGBM}}/t_{\text{FastC-GBM}}$ quantify the training time improvement of FastC-GBM relative to XGBoost and LightGBM, respectively.

### 5.1 Synthetic data

**Formulation.** To evaluate the performance of the tree-based methods, we generate a multiclass synthetic dataset in which the class label depends on a mixture of linear and nonlinear feature effects. For each simulation, we sample $N$ observations $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ as follows. The $P$-dimensional feature vector $X \in \mathbb{R}^P$ is drawn from a multivariate normal distribution $X \sim \mathcal{N}(0, \Sigma)$, where $\Sigma_{ii} = 1$ for all $i \in [P]$ and $\Sigma_{ij} = \rho$ for all $i \neq j$. Given $X$, the label $Y$ follows a categorical distribution, $Y \mid X \sim \text{Categorical}(\theta_1, \ldots, \theta_K)$, where

$$\theta_c = P(Y = c|X) := \frac{\exp(s_c)}{\sum_{c'=1}^K \exp(s_{c'})}, \quad \forall c = 1, \ldots, K$$

with

$$s_c = \sum_{j \in \mathcal{S}_1} a_j X_j + \sum_{j \in \mathcal{S}_2} b_j \sin\big(\pi X_j\big) + \sum_{j \in \mathcal{S}_3} c_j X_j^2 + \sum_{\substack{k,l \in \mathcal{S}_4 \\ k<l}} d_{kl} X_k X_l + \eta.$$

Here, the variable sets $\mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3$, and $\mathcal{S}_4$ correspond to linear, sinusoidal, quadratic, and pairwise interaction effects, respectively. The collection $\mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3, \mathcal{S}_4$ forms a partition of $[P]$, with cardinalities given by $|\mathcal{S}_1| = \lfloor P/2 \rfloor$, $|\mathcal{S}_2| = \lfloor P/4 \rfloor$, $|\mathcal{S}_3| = \lfloor P/8 \rfloor$, and $|\mathcal{S}_4| = P - \sum_{k=1}^3 |\mathcal{S}_k|$. Moreover, the weights $a_j, b_j, c_j, d_{kl}, \eta \sim \mathcal{N}(0, 1)$.

**Results.** For the synthetic datasets, we repeat each simulation 10 times, except for BDTKS, which is run only once due to its high training cost. The mean and standard deviation (reported in parentheses) of the training time and accuracy for all methods across the three settings with $\rho = 0.5$ are presented in Table 1–3. Additional results for the case $\rho = 0$ are provided in Appendix A.5.

From Table 1, CGCT delivers an impressive 8–22x speedup over the classical DT on synthetic data, with only a marginal trade-off in accuracy. In contrast, BDTKS does not demonstrate advantages in either training speed or accuracy. In the random forest and boosting settings, Table 2 and Table 3 demonstrate that DICS substantially accelerates training with negligible impact on accuracy (within 0.02). Specifically, CGRF is 8–30x faster than RF, while FastC-GBM is 3.6–27x faster than LightGBM and 5–50x faster than XGBoost.

| | Train time (s)↓ | | | Speedup↑ | Test Acc↑ | | |
|---|---|---|---|---|---|---|---|
| Dataset | DT | CGCT | BDTKS | $t_{DT}/t_{CGCT}$ | DT | CGCT | BDTKS |
| Helena | 1.30 | **0.12** | 470.25 | 10.83 | **0.28** | 0.26 | 0.16 |
| Spambase | 0.02 | **0.01** | 5.42 | 2.00 | **0.95** | 0.93 | 0.79 |
| Santander | 17.99 | **0.83** | 714.26 | 21.67 | **0.91** | 0.90 | 0.85 |
| CIFAR-10 | 42.63 | **3.33** | 533.91 | 12.80 | **0.34** | 0.31 | 0.29 |
| MNIST | 3.27 | **0.99** | 124.42 | 3.30 | 0.83 | 0.79 | **0.91** |
| Fashion-MNIST | 7.27 | **1.07** | 176.54 | 6.80 | **0.80** | 0.76 | **0.80** |

Table 4: Accuracy and training time (in seconds) for DT, BDTKS, and CGCT in real datasets. The ratio $t_{\mathrm{DT}}/t_{\mathrm{CGCT}}$ quantifies the training time improvement of CGCT relative to DT.

### 5.2 Real Datasets

**Real Datasets.** We use six real-world datasets spanning a variety of domains and scales, including Helena [29], Spambase [30], Santander [31], CIFAR-10 [32], MNIST [33], and Fashion-MNIST [34]. These datasets cover both tabular and image classification tasks, enabling a comprehensive evaluation of different model behaviors across heterogeneous data types. A summary of these datasets is given in Table 12 (Appendix).

Helena is a high-dimensional biological dataset from OpenML, commonly used for benchmarking feature selection and classification methods. Spambase is a classic UCI dataset for email spam detection based on word and character frequencies. The Santander dataset consists of anonymized customer transaction features for binary classification in a financial setting. CIFAR-10 is a standard image recognition benchmark containing 10 object categories with natural images, while MNIST is a widely used handwritten digit dataset for digit classification. Fashion-MNIST is a more challenging variant of MNIST that contains grayscale images of clothing items, providing a modern replacement for digit recognition benchmarks.

**Results.** The training time and accuracy of all methods across the three settings on real-world datasets are reported in Table 4–Table 6. Overall, tree-based algorithms enhanced with DICS achieve substantially improved training efficiency in these datasets. For the decision tree comparison in Table 4, CGCT is approximately 2-21x faster than DT, while its accuracy decreases slightly more than in the synthetic setting. In contrast, BDTKS is significantly slower and exhibits unstable performance across datasets. From Table 5, DICS accelerates the RF training by approximately 2.3-12.8x, while the accuracy drop remains small (up to 0.02). For boosting methods comparison in Table 6, the proposed FastC-GBM is consistently faster than LightGBM (up to 9.5x) and XGBoost (up to 18.75x). The only exception is the Santander dataset, where the performance degradation is likely due to implementation-level memory management issues on a large-scale dataset ($N = 200$k), rather than the algorithm itself. Finally, we observe that LightGBM performs unusually poorly on the Helena dataset, whereas FastC-GBM maintains stable performance, suggesting improved robustness compared to LightGBM, which is known to rely on a more aggressive splitting strategy that may affect stability in certain regimes.

## 6 Conclusion and Future Directions

We propose DICS, a novel approach for generating data-informed candidate splits in tree-based methods, resulting in a substantially reduced search space. Our theoretical analysis shows that, asymptotically, DICS yields candidate splits of comparable quality to those of classical decision trees, and therefore achieves similar predictive performance. Empirical results further demonstrate

| | Train time (s)↓ | | Speedup↑ | Test Acc↑ | |
|---|---|---|---|---|---|
| Dataset | RF | CGRF | $t_{\text{RF}}/t_{\text{CGRF}}$ | RF | CGRF |
| Helena | 1.43 (0.12) | **0.22 (0.02)** | 6.50 | **0.29** (0.00) | 0.28 (0.00) |
| Spambase | 0.09 (0.00) | **0.03 (0.01)** | 3.00 | **0.95 (0.00)** | 0.94 (0.00) |
| Santander | 22.04 (0.46) | **3.12 (0.04)** | 7.06 | **0.90 (0.00)** | **0.90 (0.00)** |
| CIFAR-10 | 55.96 (2.47) | **4.37 (0.12)** | 12.80 | **0.40 (0.01)** | 0.38 (0.01) |
| MNIST | 5.24 (0.25) | **2.28 (0.02)** | 2.30 | **0.93 (0.00)** | 0.92 (0.00) |
| Fashion-MNIST | 10.67 (0.05) | **2.40 (0.03)** | 4.45 | **0.83 (0.00)** | 0.82 (0.00) |

Table 5: Accuracy and training time (in seconds) for RF and CGRF in real datasets. The ratio $t_{\text{RF}}/t_{\text{CGRF}}$ quantifies the training time improvement of CGRF relative to RF.

| | Train time (s)↓ | | | Speedup↑ | | Test Acc↑ | | |
|---|---|---|---|---|---|---|---|---|
| Dataset | XGBoost | LightGBM | FastC-GBM | $\rho_1$ | $\rho_2$ | XGBoost | LightGBM | FastC-GBM |
| Helena | 13.01 (0.14) | 13.38 (0.15) | **5.48 (0.26)** | 2.37 | 2.44 | **0.35 (0.00)** | 0.23 (0.00) | 0.32 (0.00) |
| Spambase | 0.20 (0.00) | 0.16 (0.01) | **0.06 (0.00)** | 3.34 | 2.67 | 0.95 (0.00) | **0.95 (0.00)** | 0.94 (0.00) |
| Santander | **0.91 (0.03)** | 1.09 (0.02) | 1.12 (0.01) | 0.81 | 0.97 | **0.90 (0.00)** | **0.90 (0.00)** | **0.90 (0.00)** |
| CIFAR-10 | 52.50 (0.86) | 26.57 (0.29) | **2.80 (0.04)** | 18.75 | 9.49 | 0.45 (0.00) | **0.48 (0.00)** | 0.46 (0.00) |
| MNIST | 29.42 (0.07) | 4.33 (0.01) | **2.55 (0.04)** | 11.54 | 1.70 | 0.94 (0.00) | **0.95 (0.00)** | **0.95 (0.00)** |
| Fashion-MNIST | 22.24 (0.11) | 10.93 (0.10) | **2.85 (0.04)** | 7.80 | 3.83 | 0.86 (0.00) | **0.87 (0.00)** | 0.86 (0.00) |

Table 6: Accuracy and training time (in seconds) for XGBoost, LightGBM and FastC-GBM in real datasets. The ratios $\rho_1 = t_{\text{XGBoost}}/t_{\text{FastC-GBM}}$ and $\rho_2 = t_{\text{LightGBM}}/t_{\text{FastC-GBM}}$ quantify the training time improvement of FastC-GBM relative to XGBoost and LightGBM, respectively.

that DICS maintains comparable accuracy while significantly improving computational efficiency. Currently, the proposed approach focuses on accelerating classification tasks, which serves as the main *limitation*. An important direction for future work is to extend this framework by incorporating data-informed priors to similarly improve efficiency in regression settings.

# References


[1] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[2] Jerome H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232, 2001.

[3] Leo Breiman, Jerome Friedman, Richard A Olshen, and Charles J Stone. *Classification and regression trees*, volume 8. Chapman and Hall/CRC, 2017.

[4] Manish Mehta, Rakesh Agrawal, and Jorma Rissanen. SLIQ: A fast scalable classifier for data mining. In *International conference on extending database technology*, pages 18–32. Springer, 1996.

[5] Johannes Gehrke, Raghu Ramakrishnan, and Venkatesh Ganti. Rainforest-a framework for fast decision tree construction of large datasets. In *VLDB*, volume 98, pages 416–427, 1998.

[6] Wei-Yin Loh and Yu-Shan Shih. Split selection methods for classification trees. *Statistica sinica*, pages 815–840, 1997.

[7] David Maxwell Chickering, Christopher Meek, and Robert Rounthwaite. Efficient determination of dynamic split points in a decision tree. In *Proceedings 2001 IEEE international conference on data mining*, pages 91–98. IEEE, 2001.

[8] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006.

[9] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.

[10] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.

[11] Xiyang Hu, Cynthia Rudin, and Margo Seltzer. Optimal sparse decision trees. In *Advances in Neural Information Processing Systems*, volume 32, pages 7265–7273. Curran Associates, Inc., 2019.

[12] Rui Zhang, Rui Xin, Margo Seltzer, and Cynthia Rudin. Optimal sparse regression trees. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11172–11180, 2023.

[13] Jimmy Lin, Chudi Zhong, Diane Hu, Cynthia Rudin, and Margo Seltzer. Generalized and scalable optimal sparse decision trees. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*, volume 119, pages 6150–6160. PMLR, 2020.

[14] Gaël Aglin, Siegfried Nijssen, and Pierre Schaus. Learning optimal decision trees using caching branch-and-bound search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3146–3153, 2020.

[15] Hayden McTavish, Chudi Zhong, Reto Achermann, Ilias Karimalis, Jacques Chen, Cynthia Rudin, and Margo Seltzer. Fast sparse decision tree optimization via reference ensembles. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9604–9613, 2022.

[16] Rodrigo C Barros, André CPL R de Carvalho, Marcio R Basgalupp, and Marcos G Quiles. A clustering-based decision tree induction algorithm. In *2011 11th International Conference on Intelligent Systems Design and Applications*, pages 543–550. IEEE, 2011.

[17] Fei Wang, Quan Wang, Feiping Nie, Zhongheng Li, Weizhong Yu, and Fuji Ren. A linear multivariate binary decision tree classifier based on K-means splitting. *Pattern Recognition*, 107:107521, 2020.

[18] Hyafil Laurent and Ronald L. Rivest. Constructing optimal binary decision trees is NP-complete. *Information processing letters*, 5(1):15–17, 1976.

[19] Corrado Gini. *Variabilità e mutabilità: contributo allo studio delle distribuzioni e delle relazioni statistiche.* Tipogr. di P. Cuppini, 1912.

[20] Qing Ren Wang and Ching Y. Suen. Analysis and design of a decision tree based on entropy reduction and its application to large character set recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-6(4):406–417, 1984.

[21] Ping Li, Qiang Wu, and Christopher Burges. McRank: Learning to rank using multiple classification and gradient boosting. *Advances in neural information processing systems*, 20, 2007.

[22] Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1):21–27, 1967.

[23] Richard O. Duda, Peter E. Hart, and David G. Stork. *Pattern Classification.* Wiley-Interscience, New York, 2 edition, 2001.

[24] Christopher M. Bishop and Nasser M. Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.

[25] James B. McQueen. Some methods of classification and analysis of multivariate observations. In *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.*, pages 281–297, 1967.

[26] Georges Voronoi. Nouvelles applications des paramètres continus à la théorie des formes quadratiques. deuxième mémoire. recherches sur les parallélloèdres primitifs. *Journal für die reine und angewandte Mathematik (Crelles Journal)*, 1908(134):198–287, 1908.

[27] David Sculley. Web-scale k-means clustering. In *Proceedings of the 19th international conference on World wide web*, pages 1177–1178, 2010.

[28] David Arthur and Sergei Vassilvitskii. k-means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '07, page 1027–1035, USA, 2007. Society for Industrial and Applied Mathematics.

[29] OpenML. HELENA dataset. `https://www.openml.org/d/41169`, 2018.

[30] Mark Hopkins, Erik Reeber, George Forman, and Jaap Suermondt. Spambase. UCI Machine Learning Repository, 1999. OpenML dataset ID 44.

[31] Santander. Santander customer transaction prediction. `https://www.kaggle.com`, 2019.

[32] Alex Krizhevsky. Learning multiple layers of features from tiny images. *University of Toronto*, 2009.

[33] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[34] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

# A Technical appendices and supplementary material

In this Appendix, we provide the full details of the tree-based algorithms incorporating the DICS algorithm in Section A.1. The proof of the main theorem is presented in Section A.2. We further include a parameter sensitivity analysis in Section A.3, empirical validation of DICS Split Quality in Section A.4 as well as additional experimental results on synthetic data in Section A.5.

## A.1 Algorithmic Details

In this section, we present the detailed algorithms for tree-based methods incorporating DICS. In particular, we introduce the *Cluster-Guided Classification Tree* (CGCT), the *Cluster-Guided Random Forest* (CGRF), and the cluster-guided boosting method, referred to as the *Fast Classification Gradient Boosting Machine* (FastC-GBM).

### A.1.1 Cluster-Guided Classification Tree (CGCT)

DICS precalculates candidate splits $\mathcal{S}$, which yields two main advantages: (a) the number of candidate splits is reduced to $mP$, significantly lowering the computational cost; (b) $\mathcal{S}$ focuses on representative thresholds that capture the primary variations in the data. As a result, it is sufficient to explore only a subset of $\mathcal{S}$ to obtain high-quality splits, avoiding exhaustive evaluation. We refer to the resulting method as the Cluster-Guided Classification Tree (CGCT), summarized in Algorithm 2.

### A.1.2 Cluster-guided random forest (CGRF)

As an ensemble method, RF inherits the computational challenges of DT, since building each tree remains expensive when $N$ is large, even though each split only considers a random subset of features (e.g., $\sqrt{P}$ for classification). To further accelerate training, the DICS algorithm can be incorporated by replacing the standard decision tree construction with the Cluster-Guided Classification Tree (CGCT) for each tree in the ensemble. We note that the split-candidate dictionary $\mathcal{S}$ obtained from bootstrap samples typically varies only slightly, as the perturbation induced by resampling has limited impact on the underlying $K$-means clustering. Therefore, it is unnecessary to recompute $\mathcal{S}$ for every CGCT, which helps reduce redundant computations. We refer to this method as the Cluster-Guided Random Forest (CGRF), which is summarized in Algorithm 3.

**Algorithm 2** Cluster-Guided Classification Tree (CGCT)

**Require:** Feature matrix $\mathbf{X} \in \mathbb{R}^{P \times N}$, labels $\boldsymbol{y} \in [K]^N$, split-candidate dictionary $\mathcal{S}$ (Alg. 1), maximum depth $D$, number of sampled candidates $k$.
**Ensure:** Decision tree $T$

**Function** BUILDTREE($\mathbf{X}, \boldsymbol{y}, d$)
**if** $d \geq D$ **or** $|\boldsymbol{y}| \leq 2$ **or** $\boldsymbol{y}$ is pure **then**
  **return** leaf node with majority class in $\boldsymbol{y}$
**end if**
**(Split selection)**
Sample a subset $\mathcal{C} \subset \mathcal{S}$ with $|\mathcal{C}| = k$ (without replacement)
$(f^*, \tau^*) \leftarrow \arg\min_{(f,\tau)\in\mathcal{C}} Q(f, \tau)$
$\mathcal{S} \leftarrow \mathcal{S} / \{(f^*, \tau^*)\}$
**(Partition)**
$\mathbf{X}_L, \boldsymbol{y}_L \leftarrow \{(\boldsymbol{x}_i, y_i) : \mathbf{X}_{f^*,i} < \tau^*\}$
$\mathbf{X}_R, \boldsymbol{y}_R \leftarrow \{(\boldsymbol{x}_i, y_i) : \mathbf{X}_{f^*,i} \geq \tau^*\}$
**(Recursion)**
$T_L \leftarrow$ BUILDTREE($\mathbf{X}_L, \boldsymbol{y}_L, d+1$)
$T_R \leftarrow$ BUILDTREE($\mathbf{X}_R, \boldsymbol{y}_R, d+1$)
**return** node $(f^*, \tau^*, T_L, T_R)$
**return** $T \leftarrow$ BUILDTREE($\mathbf{X}, \boldsymbol{y}, 0$)

### A.1.3 Fast classification GBM (FastC-GBM)

For classification tasks, let $f_{t,c}(\boldsymbol{x}) = \sum_{j=1}^{M} w_{j,c}^t \mathbb{I}(\boldsymbol{x} \in R_j^t), \forall c = 1, \ldots, K$, denote the tree model at iteration $t$ for each class $c$; the resulting class scores are then transformed into probabilities via the softmax function. Each model $f_t = [f_{t,1}, \ldots, f_{t,c}, \ldots, f_{t,K}]^\top \in \mathbb{R}^K$ outputs a $K$-dimensional vector. Gradient tree boosting system then proceeds by iteratively solving, at iteration $t$, the following optimization problem based on the current prediction $\hat{\boldsymbol{y}}_i^{(t-1)}$:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{N} \ell(\boldsymbol{y}_i, \hat{\boldsymbol{y}}_i^{(t-1)} + f_t(\boldsymbol{x}_i)) + \Omega(f_t), \tag{3}$$

where the loss function is the multiclass cross-entropy

$$\ell(\boldsymbol{y}_i, \hat{\boldsymbol{y}}_i) = -\sum_{c=1}^{K} y_{ic} \log p_{ic}, \quad p_{ic} = \frac{\exp(\hat{y}_{ic})}{\sum_{l=1}^{K} \exp(\hat{y}_{il})}$$

and the regularization term is given by $\Omega(f_t) = \gamma M + \frac{1}{2}\lambda \sum_{c=1}^{K} \|w_{:,c}^t\|^2$. Taking a second-order approximation of (3) and expanding the regularization term yields

$$\tilde{\mathcal{L}}^{(t)} = \sum_{c=1}^{K} \sum_{j=1}^{M} [(\sum_{i\in\mathcal{I}_j} g_{ic}) w_{j,c} + \frac{1}{2}(\lambda + \sum_{i\in\mathcal{I}_j} h_{ic}) w_{j,c}^2] + \gamma M, \tag{4}$$

where $\mathcal{I}_j = \{i : \boldsymbol{x}_i \in R_j^t\}$ denote the instance set of leaf $j$ and $g_{ic}, h_{ic}$ are the first and second order gradient statistics on the loss function for each class $c$, whose formulas are given by

$$g_{ic} = p_{ic} - y_{ic}, \quad h_{ic} = p_{ic}(1 - p_{ic}), \quad \forall c = 1, \ldots, K.$$

**Algorithm 3** Cluster-Guided Random Forest (CGRF)

**Require:** Feature matrix $\mathbf{X} \in \mathbb{R}^{P \times N}$, labels $\boldsymbol{y} \in [K]^N$, number of trees $B$, number of clusters $m$, maximum depth $D$, number of sampled candidates $k$, feature subset size $\phi_p = \lfloor \sqrt{P} \rfloor$

**Ensure:** Forest $\mathcal{F} = \{T_b\}_{b=1}^{B}$

**Initialize:** $\mathcal{S} \leftarrow \mathrm{DICS}(\mathbf{X}, m)$
**for** $b = 1$ to $B$ **do**
  Draw a bootstrap sample $(\mathbf{X}^{(b)}, \boldsymbol{y}^{(b)})$ of size $N$
  Sample a feature subset $\mathcal{J}_b \subset [P]$ with $|\mathcal{J}_b| = \phi_p$
  Construct restricted dictionary $\mathcal{S}_b \leftarrow \{T_f \in \mathcal{S} : f \in \mathcal{J}_b\}$
  Grow tree $T_b \leftarrow \mathrm{CGCT}(\mathbf{X}^{(b)}, \boldsymbol{y}^{(b)}, \mathcal{S}_b, D, k)$
**end for**
**return** $\mathcal{F} = \{T_b\}_{b=1}^{B}$

The optimal leaf weights for each region $R_j$ and each class $c$, obtained by minimizing (4), are

$$w_{j,c}^{*} = -\frac{\sum_{i \in \mathcal{I}_j} g_{ic}}{\lambda + \sum_{i \in \mathcal{I}_j} h_{ic}}, \quad \forall c = 1, \ldots, K. \tag{5}$$

Let $\mathcal{I}_L$ and $\mathcal{I}_R$ denote the instance sets of the left and right child nodes induced by a candidate split $(f, \tau)$, and let $\mathcal{I} = \mathcal{I}_L \cup \mathcal{I}_R$. Then the loss reduction is given by

$$Q(f, \tau) = \frac{1}{2} \sum_{c=1}^{K} \left[ \frac{(\sum_{i \in \mathcal{I}_L} g_{ic})^2}{\lambda + \sum_{i \in \mathcal{I}_L} h_{ic}} + \frac{(\sum_{i \in \mathcal{I}_R} g_{ic})^2}{\lambda + \sum_{i \in \mathcal{I}_R} h_{ic}} - \frac{(\sum_{i \in \mathcal{I}} g_{ic})^2}{\lambda + \sum_{i \in \mathcal{I}} h_{ic}} \right] - \gamma. \tag{6}$$

which can be used as evaluating the quality of the candidate split $Q(f, \tau)$.

By using DICS and incorporating the Gradient-based One-Side Sampling (GOSS) technique [10] and column (feature) subsampling technique [9, 10], our **Fast C**lassification **G**radient **B**oosting **M**achine (*FastC-GBM*) is summarized in Algorithm 4.

## A.2 Technical Proofs

*Proof of Theorem 1.* This proof consists of two parts. In Part (i), we show that under the density regularity condition, $\delta = d_H(\mathcal{P}_{f,\bar{\tau}_*^{(f)}}, \mathcal{P}_{f,\tilde{\tau}_*^{(f)}}) = \mathcal{O}_P(\sqrt{N})$. In Part (ii), we establish that the difference between the two optimal gain values is bounded.

***Part (i).*** Fix a feature $f$, and for notational simplicity write $\tau := \tau^{(f)}$. We first establish the convergence rate of the empirical centroids $\hat{\mu}_c$ to the population means $\mu_c$. Assume the feature $f$ is bounded in $[a, b]$. By Hoeffding's inequality, for each class $c$ and any $\varepsilon > 0$,

$$\mathbb{P}\big(|\hat{\mu}_c - \mu_c| \geq \varepsilon\big) \leq 2 \exp\left(-\frac{2 n_c \varepsilon^2}{(b-a)^2}\right),$$

where $n_c$ is the number of samples in class $c$. Since $n_c \asymp N$, it follows that with probability at least $1 - \gamma$,

$$|\hat{\mu}_c - \mu_c| \leq (b-a) \sqrt{\frac{\log(2/\gamma)}{2 n_c}} = \mathcal{O}\left(\frac{1}{\sqrt{N}}\right).$$

In DICS, the candidate split is defined as $\tilde{\tau} = \zeta(\hat{\mu}_{c_1}, \hat{\mu}_{c_2}) = \frac{w_1 \hat{\mu}_{c_1} + w_2 \hat{\mu}_{c_2}}{w_1 + w_2}$, where $w_1 = \hat{\sigma}_{c_2}$ and $w_2 = \hat{\sigma}_{c_1}$. The mapping $\zeta$ is linear and thus Lipschitz continuous with constant $L = \|\nabla \zeta\|_2 = \frac{\sqrt{w_1^2 + w_2^2}}{w_1 + w_2} \leq 1$. Hence, $|\tilde{\tau} - \tau^*| \leq L \sum_c |\hat{\mu}_c - \mu_c| = \mathcal{O}_P\Big(\frac{1}{\sqrt{N}}\Big)$.

**Algorithm 4 Fast C**lassification **G**radient **B**oosting **M**achine (FastC-GBM)

**Require:** Feature matrix $\mathbf{X} \in \mathbb{R}^{P\times N}$, labels $y \in [K]^N$, split-candidate dictionary $\mathcal{S}$ (Alg. 1), number of boosting rounds $T$, maximum depth $D$, number of sampled candidates $k$, learning rate $\eta$, GOSS large-gradient size $N_a = \lfloor a \cdot N \rfloor$, GOSS small-gradient sampling size $N_a = \lfloor b \cdot N \rfloor$, column (feature) subsampling size $N_f = \lfloor c \cdot P \rfloor$

**Ensure:** Additive multiclass boosted model $F_T(\boldsymbol{x})$.

**Initialize:** $F_{0,c}(\boldsymbol{x}_i) = \log(\hat{\pi}_c), \forall c \in [K]$, where $\hat{\pi}_c$ is the empirical class proportion.
**for** $t = 1$ to $T$ **do**
  **(GOSS step)**
  Update $g_{ic} \leftarrow p_{ic}^{(t-1)} - \mathbb{I}(y_i = c), \quad \forall i \in [N], \forall c \in [K]$.
  Update $h_{ic} \leftarrow p_{ic}^{(t-1)}(1 - p_{ic}^{(t-1)}), \quad \forall i \in [N], \forall c \in [K]$.
  $\mathcal{A} \leftarrow \{i \in [N] \mid i \in \text{Top-}(N_a)\{\|g_1\|_2, \ldots, \|g_N\|_2\}\}$.
  Sample a subset $\mathcal{B} \subset [N] \backslash \mathcal{A}$ with $|\mathcal{B}| = N_b$.
  Update $g_{ic} \leftarrow \frac{1-a}{b} g_{ic}, \quad \forall i \in \mathcal{B}, \forall c \in [K]$
  Update $h_{ic} \leftarrow \frac{1-a}{b} h_{ic}, \quad \forall i \in \mathcal{B}, \forall c \in [K]$
  **(Split selection)**
  Sample a subset $\mathcal{F}_{\text{sub}} \subset [P]$ with $|\mathcal{F}_{\text{sub}}| = N_f$
  Sample a subset $\mathcal{C} \subset \mathcal{S}' = \{(f, \tau) \in \mathcal{S} : f \in \mathcal{F}_{\text{sub}}\}$ with $|\mathcal{C}| = k$ (without replacement)
  $(f^*, \tau^*) \leftarrow \arg\min_{(f,\tau)\in\mathcal{C}} Q(f, \tau)$ where $Q(f, \tau)$ is evaluated over $\mathcal{A} \cup \mathcal{B}$ using (6).
  $\mathcal{S} \leftarrow \mathcal{S}/\{(f^*, \tau^*)\}$
  Grow one leaf-wise tree $f_t$ from the split $(f^*, \tau^*)$.
  Update model $F_t(\boldsymbol{x}_i) \leftarrow F_{t-1}(\boldsymbol{x}_i) + \eta f_t(\boldsymbol{x}_i)$.
**end for**
**return** $F_T(\boldsymbol{x})$.

Let $\tau^*$ and $\bar{\tau}^*$ denote the population and empirical minimizers of the Gini impurity function $Q(\tau)$ and $Q_N(\tau)$, respectively. For a fixed feature $f$, the split is determined by the indicator $\mathbb{I}\{X^{(f)} < \tau\}$, which defines a step function with a discontinuity at $\tau$. Hence, the empirical objective is piecewise constant and its maximizer lies on a grid of order statistics of size $1/N$. Standard results from change-point estimation imply $|\tau^* - \bar{\tau}^*| = \mathcal{O}_P\big(\frac{1}{N}\big)$.

Therefore, $|\tilde{\tau} - \bar{\tau}^*| \le |\tilde{\tau} - \tau^*| + |\tau^* - \bar{\tau}^*| = \mathcal{O}_P\Big(\frac{1}{\sqrt{N}}\Big)$, which implies that $|\tilde{\tau}^* - \bar{\tau}^*| = \mathcal{O}_P\Big(\frac{1}{\sqrt{N}}\Big)$. Define the Hamming distance

$$\delta = \sum_{i=1}^{N} \mathbb{I}\left\{X_i^{(f)} \in [\min(\bar{\tau}^*, \tilde{\tau}^*), \max(\bar{\tau}^*, \tilde{\tau}^*)]\right\}.$$

Assume the marginal density $p(x^{(f)})$ is bounded by $M$ in a neighborhood of $\tau^*$. Then

$$\mathbb{E}[\delta] = N \int_{\min(\bar{\tau}^*, \tilde{\tau}^*)}^{\max(\bar{\tau}^*, \tilde{\tau}^*)} p(x^{(f)})\, dx \le NM|\tilde{\tau}^* - \bar{\tau}^*|.$$

Using $|\tilde{\tau}^* - \bar{\tau}^*| = \mathcal{O}_P(N^{-1/2})$, we obtain $\mathbb{E}[\delta] = \mathcal{O}(\sqrt{N})$. By Markov's inequality, for any $\varepsilon > 0$, $\mathbb{P}(\delta > \varepsilon\sqrt{N}) \le \frac{\mathbb{E}[\delta]}{\varepsilon\sqrt{N}} \to 0$, as $N \to \infty$. Hence, $\delta = \mathcal{O}_P(\sqrt{N})$, which completes the proof.

***Part (ii).*** For notational simplicity, we write $\mathcal{P} = (\mathcal{I}_L, \mathcal{I}_R)$ to denote the partition $\mathcal{P}_{f,\bar{\tau}_*^{(f)}}$, and $\mathcal{P}' = (\mathcal{I}'_L, \mathcal{I}'_R)$ to denote the partition $\mathcal{P}_{f,\tilde{\tau}_*^{(f)}}$. Then $\mathcal{P}$ has sizes $n_L = |\mathcal{I}_L|$ and $n_R = |\mathcal{I}_R|$. The class

counts within each partition are defined as $n_{L,c} = |\{i \in \mathcal{I}_L : y_i = c\}|$ and $n_{R,c} = |\{i \in \mathcal{I}_R : y_i = c\}|$ for each class $c \in [K]$.

If $d_H(\mathcal{P}, \mathcal{P}') = \delta$, then there are $\delta$ samples are reassigned, either from $\mathcal{I}_L$ to $\mathcal{I}_R$ or vice versa. More precisely, let $\delta_1$ denote the number of samples moving from $\mathcal{I}_L$ to $\mathcal{I}_R$ (referred to as *left-to-right*), and $\delta_2$ the number moving from $\mathcal{I}_R$ to $\mathcal{I}_L$ (referred to as *right-to-left*), so that $\delta_1 + \delta_2 = \delta$. Accordingly, the subset sizes become $n'_L = n_L - \delta_1 + \delta_2$ and $n'_R = n_R + \delta_1 - \delta_2$.

We next examine the class counts in the left and right subsets. For each left-to-right move, there exists a class $c$ such that $n_{L,c} \to n_{L,c} - 1$ and $n_{R,c} \to n_{R,c} + 1$. Similarly, for each right-to-left move, there exists a class $c$ such that $n_{L,c} \to n_{L,c} + 1$ and $n_{R,c} \to n_{R,c} - 1$.

For a set $\mathcal{I}$ with $|\mathcal{I}| = n$, we consider the Gini index in the form $G(\mathcal{I}) = 1 - \sum_{c=1}^{K} \frac{n_c^2}{n^2}$, where $n_c$ denotes the number of samples in $\mathcal{I}$ that belong to class $c$. Then the objective can be rewritten as

$$Q(\mathcal{P}) = \frac{n_L}{N} G(\mathcal{I}_L) + \frac{n_R}{N} G(\mathcal{I}_R) = 1 - \frac{1}{N} \left( \frac{\sum_{c=1}^{K} n_{L,c}^2}{n_L} + \frac{\sum_{c=1}^{K} n_{R,c}^2}{n_R} \right)$$

Hence the difference of two gains of $\mathcal{P}$ and $\mathcal{P}'$ can be found by

$$Q(\mathcal{P}') - Q(\mathcal{P}) = \frac{1}{N} \left( \frac{\sum_{c=1}^{K} (n'_{L,c})^2}{n'_L} - \frac{\sum_{c=1}^{K} n_{L,c}^2}{n_L} + \frac{\sum_{c=1}^{K} (n'_{R,c})^2}{n'_R} - \frac{\sum_{c=1}^{K} n_{R,c}^2}{n_R} \right) \tag{7}$$

Let $S_L = \sum_{c=1}^{K} n_{L,c}^2$ and $S_R = \sum_{c=1}^{K} n_{R,c}^2$, and define $S'_L, S'_R$ analogously. Denote: $I_1 = \frac{S'_L}{n'_L} - \frac{S_L}{n_L}, I_2 = \frac{S'_R}{n'_R} - \frac{S_R}{n_R}$. We first bound $I_1$. Write

$$I_1 = \frac{S'_L}{n'_L} - \frac{S_L}{n_L} = \frac{S'_L - S_L}{n_L} + S'_L \left( \frac{1}{n'_L} - \frac{1}{n_L} \right).$$

Each moved sample changes exactly one class count by $\pm 1$, so that $(n'_{L,c})^2 - n_{L,c}^2 = \pm 2 n_{L,c} + 1$, which contributes at most $|(n'_{L,c})^2 - n_{L,c}^2| \le 2n_L + 1 < 3n_L$. So for $\delta$ samples are moved, so that

$$|S'_L - S_L| = \left| \sum_{c=1}^{K} (n'_{L,c})^2 - \sum_{c=1}^{K} n_{L,c}^2 \right| \le \sum_{c=1}^{K} |(n'_{L,c})^2 - n_{L,c}^2| < 3\delta n_L \tag{8}$$

Moreover, one can easily show that $S'_L \le n'^2_L$. Combining the inequalities $S'_L \le n'^2_L$ and $|n'_L - n_L| \le \delta$ into (8) yields

$$|I_1| \le \frac{|S'_L - S_L|}{n_L} + \frac{|n'_L - n_L| S'_L}{n_L n'_L} < 3\delta + \delta \frac{n'_L}{n_L} = (4 + \frac{\delta}{n_L})\delta.$$

Similarly, we can show that $|I_2| \le (4 + \frac{\delta}{n_R})\delta$. Substituting the inequalities of $I_1$ and $I_2$ into (7) gives

$$|Q(\mathcal{P}) - Q(\mathcal{P}')| < (8 + \frac{\delta}{n_L} + \frac{\delta}{n_R}) \frac{\delta}{N} \le (8 + \frac{1}{\alpha(1-\alpha)} \cdot \frac{\delta}{N}) \frac{\delta}{N}, \tag{9}$$

where $\alpha = \min_f \{ \frac{n_L^{(f)}}{N}, \frac{n_R^{(f)}}{N} \}$. Finally, since the above bound holds uniformly over $f$, we have

$$\begin{aligned} \bar{Q}^* = \min_f Q(f, \bar{\tau}_*^{(f)}) &\le \min_f Q(f, \tilde{\tau}_*^{(f)}) + (8 + \frac{1}{\alpha(1-\alpha)} \cdot \frac{\delta}{N}) \frac{\delta}{N} \\ &= \tilde{Q}^* + (8 + \frac{1}{\alpha(1-\alpha)} \cdot \frac{\delta}{N}) \frac{\delta}{N} \end{aligned}$$

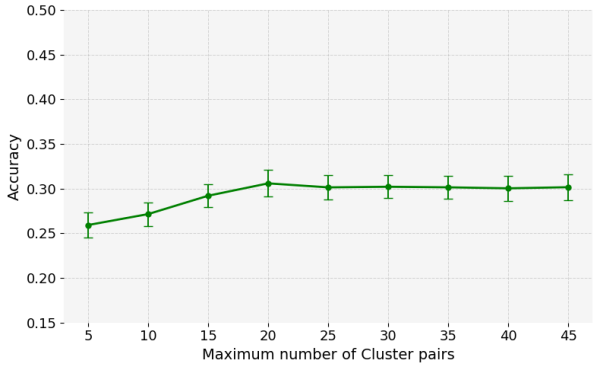
and similarly,

$$\tilde{Q}^* \leq \bar{Q}^* + (8 + \frac{1}{\alpha(1-\alpha)} \cdot \frac{\delta}{N}) \frac{\delta}{N}.$$

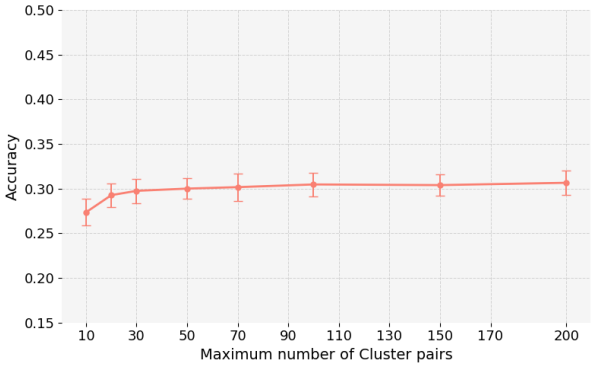
Here we use the fact that $\bar{Q}^* = \min_f Q(f, \bar{\tau}_*^{(f)}) = \min_{(f,\tau) \in \mathcal{S}_{\mathrm{DT}}} Q(f, \tau)$ and $\tilde{Q}^* = \min_f Q(f, \tilde{\tau}_*^{(f)}) = \min_{(f,\tau) \in \mathcal{S}_{\mathrm{DICS}}} Q(f, \tau)$. Combining the two inequalities above completes the proof.

□

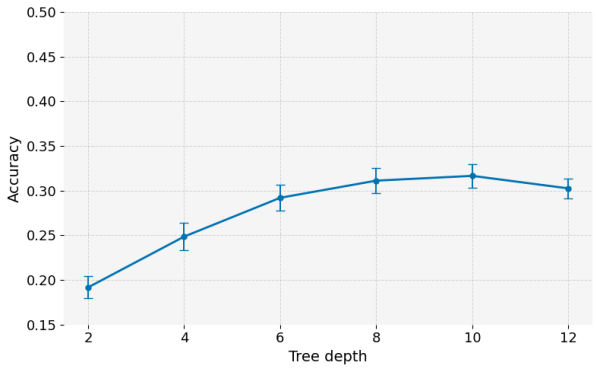
### A.3 Parameter sensitivity



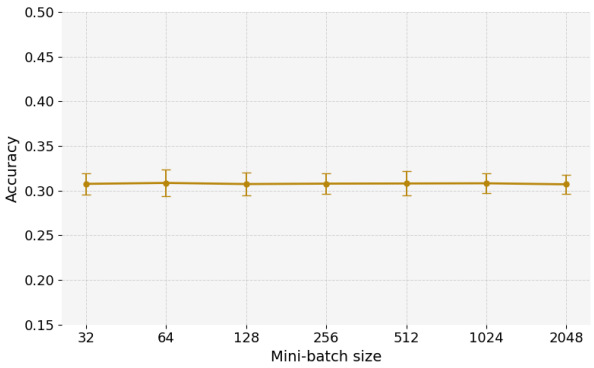

(a) Test accuracy vs. maximum number of cluster pairs ($m$) on CIFAR-10.




(b) Test accuracy vs. maximum number of feature-threshold pairs ($k$) on CIFAR-10.




(c) Test accuracy vs. maximum tree depth ($D$) on CIFAR-10.




(d) Test accuracy vs. mini-batch size ($b$) on CIFAR-10.

Figure 2: Parameter sensitivity of CGCT on CIFAR-10 with respect to (a) the maximum number of cluster pairs ($m$), (b) the maximum number of feature-threshold pairs ($k$), (c) the maximum tree depth ($D$), and (d) the mini-batch size ($b$). Each point represents the mean test accuracy over 50 runs, and error bars denote $\pm 3$ standard deviations.

We analyze the sensitivity of CGCT to four key parameters: the maximum number of cluster pairs $m$ from Algorithm 1, the length of the feature-threshold set $k$ where $|\mathcal{C}| = k$, the maximum tree depth $D$, and the mini-batch size $b$ used in mini-batch K-means. These parameters control the number of candidate splits considered during tree construction, the complexity of the resulting tree, and the computational cost of candidate generation.

Figure 2(a) shows that increasing $m$ improves accuracy at first, but the performance quickly stabilizes after a moderate value. This suggests that a small number of cluster pairs is sufficient to capture useful split candidates, while additional pairs provide limited benefit.

Figure 2(b) illustrates a similar trend for $k$. Initially, accuracy increases as more feature-threshold pairs are considered. However, beyond a specific threshold, the improvement diminishes significantly, and the curve gradually flattens. This suggests that once a sufficient number of informative candidates is included, adding more feature-threshold pairs yields only marginal benefit.

Figure 2(c) shows the effect of the maximum tree depth $D$. Accuracy improves substantially as the depth increases from shallow trees and gradually stabilizes at larger depths. The highest accuracy is obtained around $D = 8$–$10$, while increasing the depth further provides no additional improvement and results in a slight decrease at $D = 12$. This indicates that a moderate tree depth is sufficient to capture the useful structure in the candidate splits without requiring unnecessarily deep trees.

Finally, Figure 2(d) shows that CGCT is highly stable with respect to the mini-batch size $b$. Across a wide range of mini-batch sizes, from 32 to 2048, the test accuracy remains nearly unchanged. This suggests that the candidate splits generated by DICS are not highly sensitive to the mini-batch size used in K-means, and relatively small mini-batches are sufficient to obtain stable predictive performance.

Overall, these results show that CGCT is not highly sensitive to large values of the aforementioned parameters. Moderate parameter choices are sufficient to obtain stable accuracy while maintaining efficient split generation and tree construction.

### A.4 Empirical Validation of DICS Split Quality

We further evaluate the split quality of DICS from both tree-level and split-level perspectives. First, we independently grow exhaustive DT and CGCT on CIFAR-10 with maximum depth $D = 8$, allowing each method to select its own feature–threshold pair throughout tree construction. Table 7 reports the mean local gain, sample-weighted impurity reduction, and cumulative gain across depth. Overall, CGCT closely follows the impurity reduction achieved by exhaustive DT despite the two trees being grown independently. At the final splitting depth, CGCT achieves a cumulative gain of 0.123, compared with 0.127 for exhaustive DT, corresponding to 97.18% of the cumulative gain achieved by exhaustive split search.

We additionally examine the split-level behavior in Remark 1 using the synthetic-data setting by varying the sample size from $N = 1{,}000$ to $N = 20{,}000$. For each sample size, DICS and exhaustive DT independently select their best root-level feature–threshold pair, and the results are averaged over 40 independent runs. As shown in Table 8, the mean gain gap $\Delta_G$ decreases from 0.0039 to 0.0011 as $N$ increases, while $\sqrt{N}\Delta_G$ remains of comparable magnitude across the evaluated sample sizes. This behavior is consistent with the $O_P(N^{-1/2})$ split-level rate in Remark 1 and provides additional empirical evidence that the optimal DICS gain approaches that of exhaustive DT as the sample size increases.

### A.5 Additional Experiments

This section presents additional experiments on synthetic data with $\rho = 0$, where the generated features are independent of each other, as well as descriptions of the real datasets used in this paper. Additional comparisons of various algorithms are reported in Table 9–11. A summary of the real datasets is provided in Table 12.

| | Mean Local Gain $\bar{g}_d$ | | Weighted Gain $G_d^{(w)}$ | | Cumulative Gain $C_d$ | | $R_d$ (%) |
|---|---|---|---|---|---|---|---|
| Depth | DT | CGCT | DT | CGCT | DT | CGCT | |
| 1 | 0.019 | 0.019 | 0.019 | 0.019 | 0.019 | 0.019 | 100.00 |
| 2 | 0.013 | 0.012 | 0.012 | 0.012 | 0.031 | 0.031 | 99.46 |
| 3 | 0.015 | 0.014 | 0.013 | 0.013 | 0.044 | 0.043 | 98.04 |
| 4 | 0.015 | 0.015 | 0.013 | 0.013 | 0.057 | 0.057 | 99.37 |
| 5 | 0.018 | 0.017 | 0.013 | 0.013 | 0.070 | 0.070 | 99.40 |
| 6 | 0.023 | 0.021 | 0.015 | 0.014 | 0.085 | 0.084 | 98.58 |
| 7 | 0.037 | 0.030 | 0.018 | 0.017 | 0.103 | 0.101 | 97.71 |
| 8 | 0.061 | 0.051 | 0.023 | 0.022 | **0.127** | **0.123** | **97.18** |

Table 7: Depth-wise impurity reduction of independently grown exhaustive DT and CGCT on CIFAR-10 with maximum depth $D = 8$, where Depth 1 denotes the root level. For depth $d$, $\bar{g}_d = |\mathcal{I}_d|^{-1} \sum_{j \in \mathcal{I}_d} g_j$ is the mean local Gini gain, $G_d^{(w)} = \sum_{j \in \mathcal{I}_d} (n_j/N) g_j$ is the sample-weighted impurity reduction, $C_d = \sum_{r=1}^{d} G_r^{(w)}$ is the cumulative weighted gain, and $R_d = C_d^{\mathrm{CGCT}} / C_d^{\mathrm{DT}}$ is the cumulative gain retained by CGCT relative to exhaustive DT.

| $N$ | Gain Gap $\Delta_G$ | $\sqrt{N}\,\Delta_G$ |
|---|---|---|
| 1,000 | 0.0039 | 0.1227 |
| 2,000 | 0.0025 | 0.1100 |
| 5,000 | 0.0017 | 0.1183 |
| 10,000 | 0.0012 | 0.1242 |
| 20,000 | 0.0011 | 0.1492 |

Table 8: Split-level gain gap between DICS and exhaustive DT across increasing sample sizes. Results are averaged over 40 independent runs.

| $(N, P)$ | Class | Time (sec)↓ | | | Speedup↑ | ACC↑ | | |
|---|---|---|---|---|---|---|---|---|
| | | DT | BDTKS | CGCT | $t_{\mathrm{DT}}/t_{\mathrm{CGCT}}$ | DT | BDTKS | CGCT |
| 10000,1000 | 3 | 2.59 (0.02) | 31.84 | **0.18 (0.02)** | 14.39 | **0.67 (0.00)** | 0.53 | 0.65 (0.00) |
| | 5 | 2.49 (0.00) | 35.54 | **0.27 (0.02)** | 9.22 | **0.50 (0.00)** | 0.31 | 0.49 (0.00) |
| | 10 | 2.64 (0.00) | 38.21 | **0.30 (0.02)** | 8.80 | **0.46 (0.00)** | 0.30 | 0.44 (0.01) |
| 5000,7000 | 3 | 8.15 (0.09) | 29.65 | **0.63 (0.03)** | 12.94 | **0.69 (0.00)** | 0.62 | 0.68 (0.01) |
| | 5 | 8.56 (0.04) | 29.83 | **0.92 (0.05)** | 9.30 | **0.39 (0.00)** | 0.30 | 0.37 (0.01) |
| | 10 | 8.67 (0.02) | 33.40 | **1.20 (0.05)** | 7.22 | **0.34 (0.00)** | 0.24 | 0.31 (0.01) |
| 20000,5000 | 3 | 26.50 (0.33) | 118.76 | **1.20 (0.10)** | 22.08 | **0.47 (0.00)** | 0.45 | **0.47 (0.00)** |
| | 5 | 29.46 (0.04) | 125.70 | **1.69 (0.01)** | 17.43 | **0.53 (0.00)** | 0.41 | 0.51 (0.00) |
| | 10 | 29.53 (0.04) | 131.71 | **2.07 (0.07)** | 14.26 | **0.50 (0.00)** | 0.34 | 0.48 (0.00) |

Table 9: Accuracy and training time (in seconds) for DT, BDTKS, and CGCT in real datasets. The ratio $t_{\mathrm{DT}}/t_{\mathrm{CGCT}}$ quantifies the training time improvement of CGCT relative to DT. Here $\rho = 0$.

| $(N, P)$ | Class | Time (sec)↓ | | Speedup↑ | ACC↑ | |
|---|---|---|---|---|---|---|
| | | RF | CGRF | $t_{\text{RF}}/t_{\text{CGRF}}$ | RF | CGRF |
| 10000,1000 | 3 | 4.16 (0.04) | **0.34 (0.02)** | 12.23 | **0.70 (0.00)** | **0.70 (0.00)** |
| | 5 | 4.13 (0.05) | **0.39 (0.00)** | 10.59 | **0.54 (0.00)** | **0.54 (0.01)** |
| | 10 | 4.54 (0.00) | **0.50 (0.02)** | 9.08 | **0.47 (0.00)** | **0.47 (0.00)** |
| 5000,7000 | 3 | 13.67 (0.46) | **0.86 (0.07)** | 15.89 | **0.73 (0.00)** | **0.73 (0.01)** |
| | 5 | 14.13 (0.15) | **1.22 (0.03)** | 11.58 | **0.44 (0.01)** | **0.44 (0.00)** |
| | 10 | 14.51 (0.16) | **1.71 (0.07)** | 8.48 | **0.36 (0.01)** | **0.36 (0.01)** |
| 20000,5000 | 3 | 45.35 (1.86) | **1.52 (0.05)** | 29.83 | **0.51 (0.00)** | 0.50 (0.01) |
| | 5 | 57.20 (1.31) | **1.71 (0.08)** | 33.45 | **0.55 (0.00)** | **0.55 (0.00)** |
| | 10 | 54.83 (0.24) | **2.09 (0.04)** | 26.23 | **0.51 (0.00)** | **0.51 (0.00)** |

Table 10: Accuracy and training time (in seconds) for RF and CGRF in synthetic data. The ratio $t_{\text{RF}}/t_{\text{CGRF}}$ quantifies the training time improvement of CGRF relative to RF. Here $\rho = 0$.

| $(N, P)$ | Class | Time (sec)↓ | | | Speedup↑ | | ACC↑ | | |
|---|---|---|---|---|---|---|---|---|---|
| | | XGBoost | LightGBM | CGXGB | $\rho_1$ | $\rho_1$ | XGBoost | LightGBM | CGXGB |
| 10000,1000 | 3 | 1.43 (0.01) | 0.97 (0.00) | **0.30 (0.01)** | 4.77 | 3.23 | **0.71 (0.01)** | **0.71 (0.01)** | 0.70 (0.01) |
| | 5 | 2.36 (0.00) | 1.59 (0.00) | **0.37 (0.00)** | 6.38 | 4.30 | 0.54 (0.00) | **0.55 (0.01)** | **0.55 (0.01)** |
| | 10 | 4.50 (0.01) | 2.93 (0.01) | **0.44 (0.01)** | 10.23 | 6.66 | **0.48 (0.01)** | 0.47 (0.02) | 0.45 (0.01) |
| 5000,7000 | 3 | 6.73 (0.05) | 3.13 (0.06) | **0.24 (0.01)** | 28.04 | 13.04 | **0.75 (0.02)** | **0.75 (0.02)** | 0.74 (0.01) |
| | 5 | 10.81 (0.23) | 4.80 (0.15) | **0.29 (0.00)** | 37.28 | 16.55 | **0.41 (0.02)** | **0.41 (0.02)** | 0.38 (0.01) |
| | 10 | 18.16 (0.46) | 9.30 (0.59) | **0.37 (0.01)** | 49.08 | 25.13 | **0.35 (0.01)** | 0.34 (0.01) | 0.33 (0.01) |
| 20000,5000 | 3 | 9.40 (0.24) | 5.73 (0.17) | **0.68 (0.00)** | 13.82 | 8.43 | 0.54 (0.01) | **0.55 (0.01)** | 0.52 (0.01) |
| | 5 | 15.06 (0.19) | 8.13 (0.08) | **0.77 (0.00)** | 19.56 | 10.56 | **0.55 (0.01)** | **0.55 (0.01)** | 0.54 (0.01) |
| | 10 | 31.04 (0.72) | 15.10 (0.35) | **1.09 (0.03)** | 28.48 | 13.85 | **0.51 (0.01)** | **0.51 (0.01)** | 0.50 (0.01) |

Table 11: Accuracy and training time (in seconds) for XGBoost, LightGBM and FastC-GBM in synthetic data. The ratios $\rho_1 = t_{\text{XGBoost}}/t_{\text{FastC-GBM}}$ and $\rho_2 = t_{\text{LightGBM}}/t_{\text{FastC-GBM}}$ quantify the training time improvement of FastC-GBM relative to XGBoost and LightGBM, respectively. Here $\rho = 0$.

| **Dataset** | **N** | **P** | **K** |
|---|---|---|---|
| Helena | 65,196 | 27 | 100 |
| Spambase | 4,601 | 57 | 2 |
| Santander | 200,000 | 200 | 2 |
| CIFAR-10 | 60,000 | 3,072 | 10 |
| MNIST | 70,000 | 784 | 10 |
| Fashion-MNIST | 70,000 | 784 | 10 |

Table 12: Summary of datasets used in the experiments, including sample size ($N$), number of features ($P$), and number of classes ($K$).